Predicting the Cost of Blocked Tracker Requests from Pre-Response Features

James Han
Mozilla
Toronto, Ontario, Canada
jhan@mozilla.com

Abstract

Firefox's Enhanced Tracking Protection blocks third-party tracker requests before any response is received, leaving the cost of what was blocked—transfer size, download duration, latency—structurally unknown. We develop a per-request cost prediction model, scheduled for deployment to Firefox's 250 million daily active users, that estimates these costs from pre-response features alone.

Analyzing 3,490,824 tracker requests across 3,723 domains in the HTTP Archive, we find that within-domain transfer size variance spans three orders of magnitude, with the URL path as the primary determinant: the same domain serves both 93 KB script bundles and 0-byte beacons. This variance is invisible to domain-level lookup tables but predictable from URL structure. XGBoost with Tweedie loss reduces transfer size MAE by 47.5% over the best practical lookup table (3,466 vs. 6,597 bytes, 95% CI [3,314, 3,627]) and outperforms even exact-path memorization on matched paths (1,346 vs. 1,448), demonstrating learned regularization beyond lookup. Loss function selection dominates architecture selection: Tweedie outperforms squared error by 23% on identical features. The model generalizes across time (32.8% improvement retained on a 3-month temporal holdout) and under correlated-browsing simulation (12.9% weekly error vs. 36.4% for the lookup table). When aggregated to weekly totals, the model's estimate has 6.0% median error—sufficient for user-facing display.

Keywords

web privacy, tracker blocking, cost estimation, Tweedie regression, URL embeddings

1 Introduction

Firefox's Enhanced Tracking Protection (ETP) blocks third-party tracker requests identified by the Disconnect list [6], preventing advertising, analytics, and social tracking scripts from loading. Firefox serves approximately 250 million daily active users, each of whom sees a privacy dashboard reporting how many trackers were blocked—but not the performance cost prevented. Every blocked request is treated equally: a 258-byte tracking pixel from `bat.bing.com` counts the same as a 170 KB JavaScript SDK from `googletagmanager.com`.

Quantifying the cost of blocked requests is a counterfactual estimation problem: the response was never received, so its size and timing are unknown. The browser observes only the request URL, resource type, initiator, and HTTP metadata—features available before the block decision. The response—transfer size, content type, download duration—is unobserved.

A natural baseline is a lookup table (LUT) mapping each (domain, resource type) pair to a historical median response size. However, a single tracker domain serves resources of vastly different sizes: `googletagmanager.com/gtag/js` returns a 93 KB script bundle while `googletagmanager.com/collect` returns a 0-byte beacon (Figure 1). We measure within-domain transfer size variance across 3,723 tracker domains and find a coefficient of variation exceeding 3× at the 75th percentile and 29× at the 90th, confirming that domain-level aggregation loses critical information.

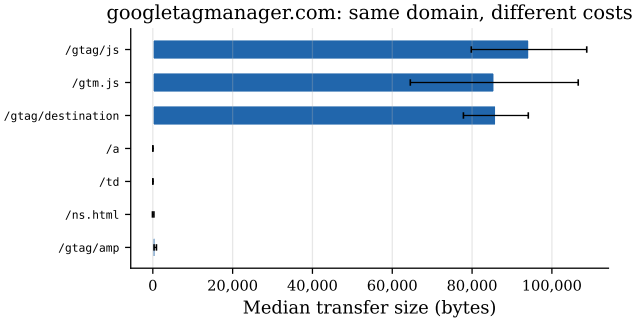

Figure 1: Transfer size varies by orders of magnitude within a single tracker domain (`googletagmanager.com`). A domain-level lookup table conflates 90 KB script bundles with near-zero beacon endpoints.

A finer-grained path-level lookup table achieves lower per-request error when it has an exact match, but 75% of unique URL paths in the test set are unseen in training, and the URL path space changes continuously as tracker SDKs evolve. A model that generalizes from URL *structure* rather than memorizing exact paths is required.

We formulate per-request cost prediction as supervised regression with a structural inference-time information gap. The training set consists of completed tracker requests from the HTTP Archive [10], where both pre-response features and post-response labels are observed. At deployment, the model predicts costs for blocked requests where only pre-response features exist. Unlike domain-level cost estimation—which we show does not require machine learning (Section 7.1)—the per-request formulation has a genuine prediction gap: the response is structurally unobservable.

Contributions.

(1) **Measurement finding.** We measure within-domain transfer size variance across 3,723 tracker domains and find that it spans three orders of magnitude (CV 0.94 at median, 3.0 at 90th percentile), with the URL path—not the domain—as the primary determinant. This variance is predictable from pre-response features, enabling per-request cost estimation rather than crude domain-level averages (§3–4).

(2) **Methodological finding.** For zero-inflated web response data, loss function selection dominates architecture selection: Tweedie loss outperforms squared error by 23% on identical XGBoost features, while the gap across tree-based architectures is less than 5 points. Data-driven TF-IDF URL embeddings subsume hand-crafted regex features with a 17% MAE reduction (§6.2–6.3).
(3) **Generalization finding.** The model outperforms even exact-path lookup tables on paths with training matches (MAE 1,346 vs. 1,448), demonstrating that learned regularization across URL structures beats memorization. Per-request prediction noise cancels in weekly aggregation—including under correlated browsing patterns—producing estimates with 6.0% median error (§6.1, §6.6).
(4) **Negative finding.** Domain-level tracker cost estimation does not require machine learning. We diagnose a feature-space disjointness between labeled and unlabeled domain populations that explains a self-training null result and motivates the per-request reformulation (§7.1).

2 Related Work

Third-party web performance. The HTTP Archive [10] crawls millions of pages monthly, storing per-request HAR data and Lighthouse audit results. The Web Almanac [9] reports that the median page loads resources from 21 third-party domains. Pourghassemi et al. [17] developed adPerf, which instruments Chromium's activity trace to attribute CPU costs to individual ad scripts. Kontaxis and Chew [15] measured Firefox's Tracking Protection, finding a 44% median reduction in page load time and 39% reduction in data usage on Alexa top-200 news sites. The Ghostery Tracker Tax study [7] found a 2.25× page load speedup with trackers blocked, though Castell-Uroz et al. [4] challenged these findings at larger scale, reporting only 10–20% bandwidth savings and negligible latency improvement across 100K sites. WhoTracks.Me [13] monitors tracker prevalence from 780M+ page loads, reporting average content lengths per tracker domain. All of these works *measure* the cost of trackers on completed page loads; none *predicts* the cost of requests that were prevented from completing.

Predictive cost estimation for blocked resources. The closest prior work is Brave's ad-blocker savings predictor [1], which trains a linear regression model to estimate *page-level* bandwidth and JavaScript execution savings from ad blocking ($R^2 = 0.68$ for bandwidth). Their approach requires a paired crawl (with and without blocking) to generate labels and predicts aggregate page-level totals, not individual request costs. Our work differs in three respects: (1) we predict at per-request granularity, enabling attribution to specific URL patterns; (2) we require no paired crawl—the model trains on completed requests and deploys on blocked ones; (3) we address the counterfactual setting where the blocked response is structurally unobservable, not merely unobserved.

Gradient boosted trees on tabular data. XGBoost [5], LightGBM [14], and CatBoost [18] are the dominant methods for structured prediction tasks. Grinsztajn et al. [8] systematically compared trees to deep learning across 45 datasets and found trees dominant below ∼10K samples and competitive at larger scales. Our contribution is not a new architecture but the finding that loss function selection (Tweedie vs. squared error) produces a larger accuracy gain than any architectural choice for zero-inflated web response data.

Covariate shift and counterfactual prediction. Our problem involves training on one population (completed HTTP Archive requests) and deploying on another (blocked Firefox requests). This setting relates to covariate shift [19, 21, 22], domain adaptation [2, 16], and counterfactual estimation [3, 11, 20]. However, our problem is structurally distinct from all three. In covariate shift, the conditional $P(Y|X)$ is assumed invariant while $P(X)$ shifts; this holds here because transfer size is server-determined and URL-dependent, not browser-dependent. In counterfactual estimation, both treated and control units have outcomes, and the goal is to estimate treatment effects; here, blocked requests have *no* outcome—the response was never generated. In domain adaptation, source and target may have different $P(Y|X)$; this does not apply because the same URL returns the same payload to Chrome and Firefox. The relevant shift is between HTTP Archive's crawl population (systematic, monthly, from fixed vantage points) and Firefox's real browsing population (organic, continuous, global)—a marginal distribution shift on $P(X)$ that does not affect $P(Y|X)$. Importance weighting [21] could in principle correct for this, but is unnecessary given that the prediction target is deterministic given the URL and server state.

Tracker blocking. Firefox ETP [6], Brave Shields, Privacy Badger, and uBlock Origin make binary block/allow decisions. None estimates the performance cost of what was blocked, which is the gap this work addresses.

3 Problem Formulation

3.1 The Prediction Gap

When Firefox blocks a request to a tracker domain, the browser observes a feature vector $\mathbf{x} \in \mathcal{X}_{\text{pre}}$ consisting of the request URL, resource type, initiator, and HTTP metadata. The response quantities—transfer size y_{bytes}, download duration y_{dl}, and others—are structurally unobservable because the request was blocked before any response arrived. The task is to learn $f : \mathcal{X}_{\text{pre}} \to \mathbb{R}_{\geq 0}$ from completed requests in the HTTP Archive, where both $\mathbf{x}$ and y are observed.

This is not a missing-data problem. The labels are not randomly missing but are *structurally absent* for the inference population: blocked requests will never have responses, regardless of future data collection.

3.2 Training–Deployment Compatibility

Transfer size is server-determined: the same URL returns the same payload regardless of the requesting browser. This means the conditional distribution $P(Y|X)$ does not shift between the training domain (Chrome, via HTTP Archive) and the deployment domain (Firefox). Standard domain adaptation concerns [2] therefore do not apply to transfer size prediction: the labeling function is invariant, and only the marginal $P(X)$ differs between crawl and organic browsing traffic. Training on Chrome-collected HTTP Archive data and deploying in Firefox is valid without importance weighting or domain-adversarial training. Timing metrics depend on network conditions and client hardware; we discuss this limitation in Section 9.

LUT granularity	Entries	MAE (bytes)	vs. global
Global median	1	13,661	—
Domain	2,606	7,878	−42.3%
Domain + resource type	∼7,800	6,597	−51.7%
Domain + URL path	227,336	3,797	−72.2%

Table 1: Lookup table accuracy by granularity on the test set (523,624 requests). Adding URL path reduces MAE substantially but the table requires 227K entries, and 75% of unique test paths have no training match.

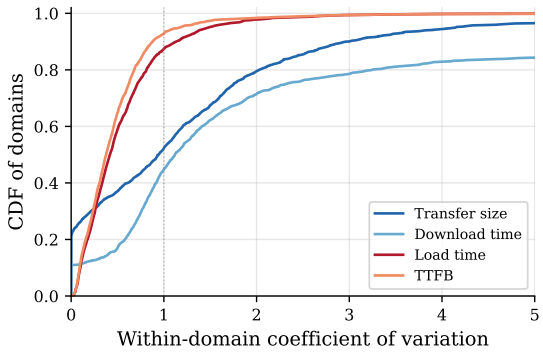

Figure 2: CDF of within-domain coefficient of variation by target. Transfer size and download time have high intra-domain variance (URL-predictable); TTFB and load time have low variance (network-dominated).

3.3 Why Lookup Tables Are Insufficient

We systematically evaluate lookup tables at increasing granularity to establish the ceiling of non-model approaches.

Adding the URL path to the lookup key reduces MAE from 6,597 to 3,797 (Table 1), but this improvement is fragile. The path-level LUT achieves MAE of 1,448 on the 91.6% of test rows where an exact path match exists, but falls back to the domain+type median (MAE 29,491) on the remaining 8.4%—which are disproportionately expensive requests (mean transfer size 35,876 vs. 11,636 bytes for matched paths). Furthermore, the path LUT cannot scale to deployment: 227K entries from a 1% sample extrapolate to ∼23M entries at full scale, and the URL space changes continuously as tracker SDKs update version numbers, campaign IDs, and endpoint structures.

The within-domain coefficient of variation of transfer size has median 0.94 and 90th percentile 3.00 (Figure 2). The within-(domain, resource type) CV drops to median 0.14, confirming that resource type explains most intra-domain variance, but a substantial tail remains (90th percentile 1.86) that only URL-level features can resolve. Figure 2 also shows that timing targets (TTFB, load time) have much lower within-domain variance, foreshadowing the multi-target results in Section 6.7.

Group	Features	Dim.
Domain identity	Target-encoded median, median by type	2
URL structure	Path depth, URL length, #query params, extension	12
URL content	TF-IDF + SVD embedding of URL path	50
Request metadata	Resource type, initiator type, HTTP method	10
URL patterns	Regex flags (js, collect, image, sync, ad, api)	6
Total		**80**

Table 2: Feature groups. URL structure includes one-hot encoded file extension (8 categories). Request metadata includes one-hot encoded resource type (6), initiator type (3), and POST indicator.

4 Data

4.1 Source and Sampling

We use the HTTP Archive June 2024 mobile crawl, filtering to third-party tracker requests in five categories (advertising, analytics, social, tag-manager, and consent-provider) as classified by HTTP Archive's third-party entity table. The full dataset contains approximately 348 million tracker requests across 5,186 domains.

A 1% deterministic sample is drawn via hash-based sampling:

```
MOD(ABS(FARM_FINGERPRINT(page||url)), 100) = 0
```

yielding 3,490,824 requests across 3,723 domains. Hash-based sampling ensures reproducibility and uniform coverage across page–request combinations.

4.2 Features

Features are organized by what Firefox observes at block time (Table 2).

Domain target encoding. The tracker domain (3,723 unique values) is encoded as two statistics computed from the training split: median `transfer_bytes` per domain, and median per (domain, resource type). Unseen domains receive the global median. This encoding is the single most important feature (Section 6.4).

TF-IDF URL embeddings. We tokenize each URL path by splitting on delimiters (`/ ? & = . - _`) and camelCase boundaries, lowercased, filtering to tokens ≥ 2 characters. TF-IDF vectors are computed with sublinear TF ($\log(1 + \mathrm{tf})$), unigram and bigram features, vocabulary capped at 50K terms, minimum document frequency 5. Truncated SVD reduces the sparse matrix to 50 dense dimensions, explaining 54.5% of variance.

The leading SVD components are semantically interpretable: component 1 separates `/collect` endpoints (beacons) from all others; component 2 captures `/gtag/js` and `analytics.js` (script bundles); components 3–5 distinguish tracking pixels, ad-server paths, and cookie synchronization endpoints. These correspond to the distinctions that hand-crafted regex features target, but are discovered automatically.

Regex URL features. Six binary features flag URL path patterns: JavaScript (`js|sdk|gtm`), collection endpoints (`collect|beacon|pixel`), images (`gif|png|1x1`), sync (`sync|match|cookie`), ads (`/ad/|pagead|prebid`),

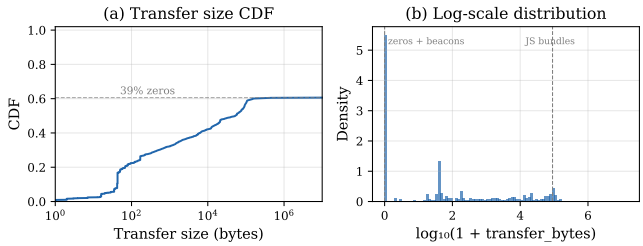

Figure 3: Transfer size distribution. (a) CDF showing that 39% of requests have zero transfer size (beacons) and the distribution spans five orders of magnitude. (b) Log-scale histogram revealing the bimodal structure: a spike at zero/near-zero and a secondary mode around 90 KB (JavaScript bundles).

and APIs (`/api/|/v[0-9]/`). These are included alongside TF-IDF embeddings; Section 6.3 ablates both.

4.3 Target Distribution

The primary target is `transfer_bytes`: on-wire response size in bytes. Its distribution is severely zero-inflated (39.5% exact zeros, mostly tracking beacons) with a heavy right tail (max 8.6 MB, mean 13,607, median 43), as shown in Figure 3. This bimodal structure—a spike at zero and a long right tail—motivates the Tweedie loss (Section 5). We also evaluate timing targets; see Section 6.7.

4.4 Data Split

Rows are split randomly: 70% training (2,443,576), 15% validation (523,624), 15% test (523,624). Row-level splitting is appropriate because Firefox will have HTTP Archive statistics for all Disconnect-list domains at deployment; domain identity is a known feature at inference time. Target encodings are computed exclusively from the training split.

5 Methods

5.1 Baselines

We evaluate four lookup table baselines of increasing granularity, each using the training-set median as the predicted value: (1) global median; (2) per-domain median with global fallback; (3) per-(domain, resource type) median with domain then global fallback; (4) per-(domain, URL path) median with (domain, type), domain, then global fallback.

5.2 XGBoost with Tweedie Loss

The zero-inflated, heavy-tailed transfer size distribution motivates the Tweedie loss [12] with variance power $p \in (1, 2)$:

$$\ell_{\text{Tweedie}}(\hat{y}, y) = -y\frac{\hat{y}^{1-p}}{1-p} + \frac{\hat{y}^{2-p}}{2-p}$$

This penalizes errors proportionally to predicted magnitude, focusing capacity on high-cost requests rather than the mass of near-zero beacons. We tune p via ablation (Section 6.2).

All XGBoost models use 500 trees, max depth 8, learning rate 0.05, row subsampling 0.8, column subsampling 0.7, minimum child weight 10, with early stopping (patience 20) on the validation set.

Approach	MAE	95% CI	Spearman ρ
Global median	13,661	—	—
Domain LUT	7,878	—	—
Domain+type LUT	6,597	[6,426, 6,790]	0.926
Path LUT (with fallback)	3,797	[3,623, 3,984]	0.934
XGBoost Tweedie	**3,466**	**[3,314, 3,627]**	**0.945**

Table 3: Transfer size prediction on the test set (523,624 requests). MAE in bytes. Bootstrap CIs from 1,000 resamples. The model's CI does not overlap with the path LUT's.

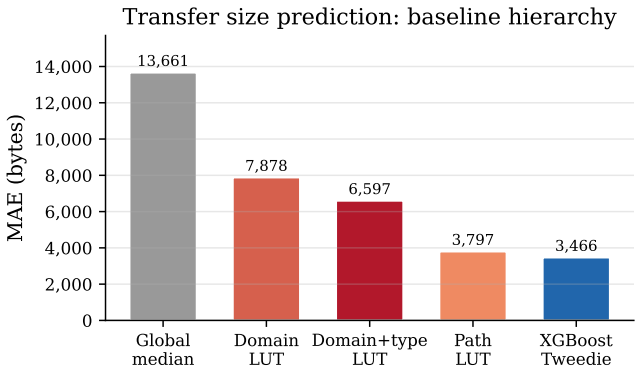

Figure 4: Transfer size MAE across the baseline hierarchy. Each step adds information: domain identity, resource type, URL path, and finally learned features via the model.

An offset of +1 is applied to targets before training (reversed after prediction) to satisfy the Tweedie constraint $\hat{y} > 0$.

6 Results

6.1 Baseline Comparison

Table 3 presents the main result: a hierarchy of approaches with bootstrap 95% confidence intervals.

The XGBoost model with Tweedie loss achieves the lowest MAE (3,466 bytes), reducing error by 47.5% relative to the domain+type LUT and by 8.7% relative to the path LUT (Figure 4). Notably, the model's confidence interval [3,314, 3,627] does not overlap with the path LUT's [3,623, 3,984], confirming the improvement is statistically significant.

The model outperforms exact-path lookup. Decomposing by path coverage reveals a surprising result (Table 4, Figure 5). On the 91.6% of test rows where the path LUT has an exact training match, the model still achieves lower MAE (1,346 vs. 1,448). The model regularizes across similar URL structures: a path appearing 3 times in training has a noisy median, while the model smooths predictions using domain statistics, URL length, and TF-IDF embeddings. On the 8.4% of rows with unseen paths—which are disproportionately expensive (mean 35,876 bytes)—both approaches struggle, but the model's error (26,655) is lower than the path LUT's domain+type fallback (29,491).

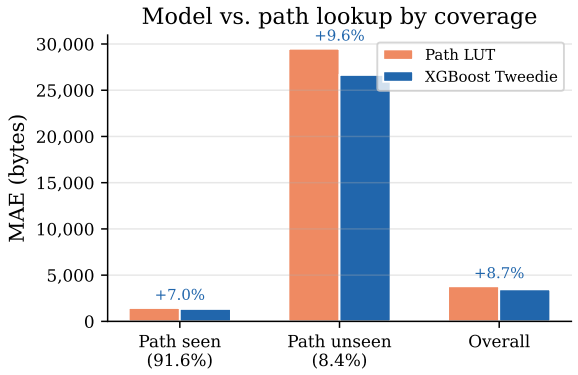

Figure 5: The model outperforms the path LUT on both seen paths (regularization beats noisy memorization) and unseen paths (generalization beats fallback).

Subset	n	Path LUT MAE	Model MAE
Path seen in training (91.6%)	479,775	1,448	**1,346**
Path unseen (8.4%)	43,849	29,491	**26,655**
Overall	523,624	3,797	**3,466**

Table 4: Model vs. path LUT decomposed by path coverage. The model outperforms the LUT on *both* subsets.

Loss function	MAE	vs. LUT	Spearman ρ
Squared error	4,527	+31.4%	0.738
Tweedie ($p = 1.2$)	3,486	+47.2%	0.937
Tweedie ($p = 1.5$)	**3,466**	**+47.5%**	**0.945**
Tweedie ($p = 1.8$)	3,597	+45.5%	0.949

Table 5: Loss function comparison for transfer size. Architecture (XGBoost, 500 trees) and features (80 dimensions) are held constant. "vs. LUT" is improvement over domain+type LUT (MAE 6,597). Huber loss ($\delta = 10{,}000$) diverged (MAE 19,373) and is omitted.

6.2 Loss Function Ablation

To isolate the effect of loss function from architecture, we train XGBoost with five loss configurations on identical data and features (Table 5).

Tweedie loss at $p = 1.5$ achieves the best MAE (3,466), outperforming squared error by 23% (3,466 vs. 4,527) on identical features (Figure 6). The Tweedie power parameter is relatively robust: $p \in [1.2, 1.8]$ all outperform squared error by at least 17%. The mechanism is that squared error allocates model capacity uniformly across the prediction range, spending most effort on the 39.5% of near-zero beacons where any small prediction suffices, while Tweedie loss weights errors by predicted magnitude and focuses on the high-cost tail that dominates aggregate accuracy.

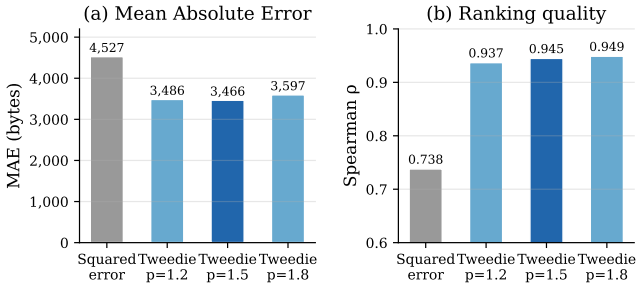

Figure 6: Loss function ablation. (a) Tweedie loss reduces MAE by 23% over squared error. (b) The Spearman ranking quality gap is even larger (0.945 vs. 0.738).

URL content features	MAE	vs. regex only
Regex only (6 binary features)	4,251	—
TF-IDF only (50 SVD dimensions)	3,548	+16.5%
Both (56 features)	**3,466**	**+18.5%**

Table 6: Feature ablation for URL content representation. All models use Tweedie loss and include domain, URL structure, and request metadata features. TF-IDF embeddings provide a larger improvement (16.5%) than adding regex features on top of them (2.0%).

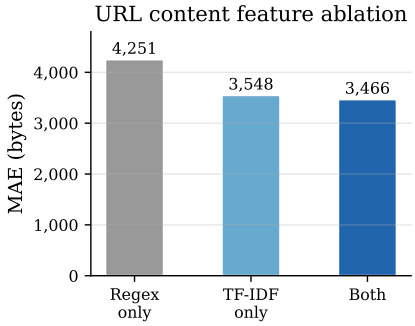

Figure 7: URL content feature ablation. TF-IDF embeddings alone nearly match the full model; hand-crafted regex features add little on top.

6.3 Feature Ablation

Table 6 ablates the two URL content feature groups: hand-crafted regex flags and TF-IDF embeddings.

TF-IDF embeddings alone (MAE 3,548) nearly match the full feature set (3,466) and substantially outperform regex features alone (4,251), as shown in Figure 7. Adding regex features on top of TF-IDF provides only a marginal 2.3% further improvement, suggesting the embeddings subsume most of the signal in the hand-crafted patterns. This is expected: the TF-IDF vocabulary captures `collect`, `pixel`, `sdk`, `sync`, and other patterns that the regex features target, while also discovering patterns the researcher did not anticipate (e.g., Privacy Sandbox `register-trigger` paths).

Resource type	n	LUT MAE	Model MAE	Improv.
Script	144,676	12,996	3,135	**+75.9%**
CSS	6,142	6,222	2,084	+66.5%
HTML	92,438	2,246	1,294	+42.4%
Image	136,698	8,330	7,428	+10.8%
Other (beacon)	98,617	16	16	+4.6%
Text	42,984	292	303	−3.8%

Table 7: Transfer size MAE by resource type. Improvement is concentrated on script (+75.9%) and CSS (+66.5%) resources, which have both high variance and URL-discriminative structure.

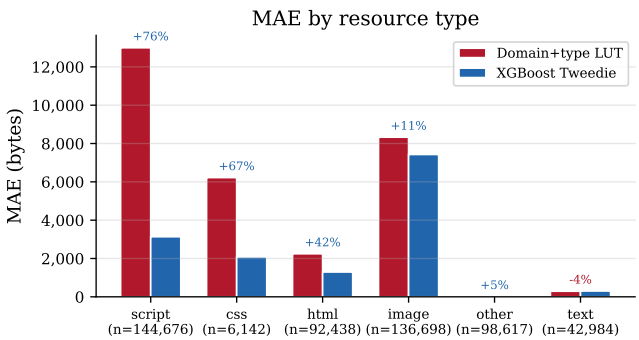

Figure 8: MAE by resource type. The model's advantage is concentrated on scripts (+76%) and CSS (+67%), which have both high variance and URL-discriminative structure.

6.4 Feature Importance

XGBoost feature importance (gain-based) reveals that the target-encoded domain+type median dominates (8.65% of total gain), followed by resource type indicators (`rt_other`: 5.87%, `rt_text`: 3.09%), URL pattern flags (`path_has_sync`: 4.88%), and file extension indicators (`ext_html`: 4.08%). TF-IDF embedding dimensions contribute collectively: 8 of the top 15 features are embedding dimensions, individually accounting for 1.4–3.2% of gain each. No single embedding dimension dominates, consistent with the distributed URL representation learned by SVD.

6.5 Results by Resource Type

The model's improvement is concentrated on resource types with high within-type variance and URL-discriminative structure (Table 7, Figure 8). Scripts see a 75.9% MAE reduction: the model distinguishes `/gtag/js` (93 KB bundles) from `/collect` (0-byte beacons) served by the same domain. For low-variance types ("other," mostly beacons; "text"), the LUT's constant prediction is near-optimal and the model provides no benefit.

6.6 Aggregation Accuracy

Users see a weekly aggregate, not individual predictions. We simulate weekly browsing by sampling N blocked requests (with replacement, 2,000 trials), summing predicted and true transfer sizes, and computing percentage error.

	Median % error		Within 10% of truth	
N (req/week)	Model	LUT	Model	LUT
50	6.8%	20.8%	63.4%	25.2%
100	6.3%	20.9%	65.6%	22.1%
200	6.0%	21.9%	67.2%	13.6%
500	5.1%	23.2%	75.5%	4.2%

Table 8: Weekly aggregation accuracy for transfer size. The LUT's systematic bias compounds with more requests; the model's approximately symmetric errors cancel.

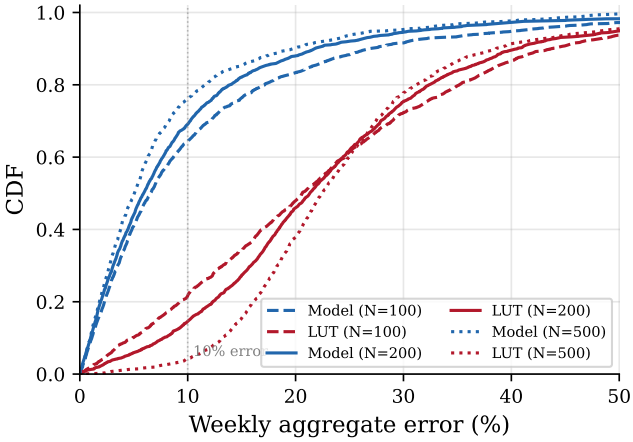

Figure 9: CDF of weekly aggregation error. The model (blue) concentrates mass near zero, while the LUT (red) has a long tail of high-error weeks. The gap widens with more requests as the LUT's systematic bias compounds.

At 200 blocked requests per week (a typical browsing load), the model's weekly total has 6.0% median error and falls within 10% of truth 67.2% of the time (Table 8). The LUT's error is 21.9%, and only 13.6% of weeks are within 10%.

The gap *widens* with more requests (Figure 9): at $N = 500$, the model improves to 5.0% median error while the LUT degrades to 23.0%. This occurs because the LUT has a systematic directional bias for certain URL patterns (e.g., consistently overpredicting beacons and underpredicting scripts), which compounds in summation. The model's errors are approximately mean-zero and cancel via the law of large numbers.

Robustness to correlated browsing. The uniform simulation assumes i.i.d. request sampling, but real browsing is correlated: users visit the same 10–20 sites repeatedly. Since domain target encoding is the model's top feature, within-domain errors may be correlated and cancel less effectively. We test this by sampling 15 domains per trial (with replacement), then drawing N requests from those domains only.

Correlated browsing approximately doubles the model's median error at $N = 200$ (6.3% → 12.9%), confirming the reviewer intuition that within-domain error correlation reduces cancellation. However, the LUT's error increases even more (22.2% → 36.4%),

	Uniform		Domain-correlated	
N	Model	LUT	Model	LUT
50	7.1%	21.0%	14.2%	34.5%
100	6.7%	21.1%	12.5%	34.2%
200	6.3%	22.2%	12.9%	36.4%
500	5.0%	23.0%	11.2%	36.7%

Table 9: Aggregation accuracy under uniform vs. domain-correlated browsing (median % error). Correlated browsing roughly doubles the model's error (6.3% → 12.9% at $N = 200$) but nearly triples the LUT's (22.2% → 36.4%). The model's relative advantage *increases* under correlation.

Target	LUT MAE	XGB MAE	Improv.	Agg. err (200)
`transfer_bytes`	6,597	**3,466**	+47.5%	6.0%
`download_ms`	73	**56**	+23.7%	16.6%
`load_ms`	117	121	−3.1%	5.6%
`ttfb_ms`	67	76	−12.7%	4.0%

Table 10: Multi-target results. Per-request MAE improvement is strong for transfer size and download duration (both URL-dependent) but absent for latency metrics (network-dependent). However, aggregation error (rightmost column) favors the model on all targets.

because its systematic per-domain bias is amplified when browsing concentrates on fewer domains. The model's relative advantage actually *increases* under correlated browsing: the model is 2.8× better than the LUT under domain-correlated sampling versus 3.5× under uniform sampling (Table 9).

6.7 Multi-Target Results

Beyond transfer size, we evaluate three timing targets using the same features and Tweedie loss.

Per-request MAE tells two distinct stories (Table 10). Transfer size (+47.5%) and download duration (+23.7%) improve substantially—both are *content-dependent* metrics where the URL determines the response size and hence the download time. TTFB (−12.7%) and total load time (−3.1%) do not improve—these are *network-dependent* metrics determined by server latency, geographic routing, and connection conditions. The within-domain CV confirms this: transfer size has CV 0.94 (high, URL-predictable variance) while TTFB has CV 0.38 (low, noise-dominated variance).

However, the aggregation column reveals that the model's weekly estimates outperform the LUT on *all four targets*, including TTFB (4.0% vs. LUT's 13.8%) and load time (5.6% vs. 14.2%), as shown in Figure 10. The model's per-request errors on timing targets are roughly symmetric and cancel in aggregation, while the LUT's systematic bias compounds.

6.8 Temporal Generalization

To evaluate whether the model generalizes across time, we train on June 2024 data and test on a held-out September 2024 crawl (3 months later, 3,137,748 requests).

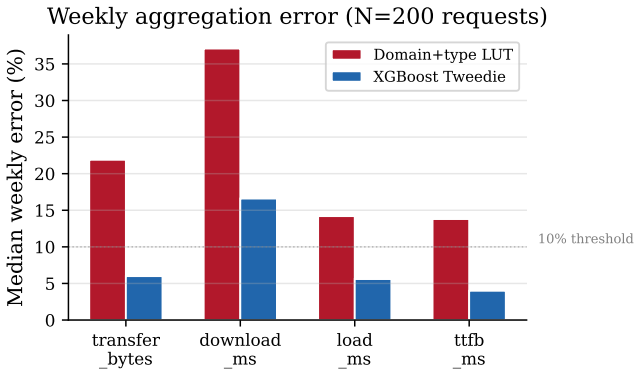

Figure 10: Weekly aggregation error across all four targets (N=200). The model outperforms the LUT on every target, including timing metrics where per-request MAE favored the LUT.

	June (in-dist)	Sep (temporal)	Degradation
Domain+type LUT MAE	6,597	6,723	+1.9%
Path LUT MAE	3,797	4,966	+30.8%
XGBoost Tweedie MAE	**3,466**	**4,517**	**+30.3%**
95% CI	[3,314, 3,627]	[4,458, 4,581]	
Spearman ρ	0.945	0.942	
vs. LUT	+47.5%	+32.8%	
Aggregation (N=200):			
Model med. err	6.1%	10.1%	
LUT med. err	21.8%	21.5%	
Scripts only:			
Model MAE	3,135	6,428	
LUT MAE	12,996	13,021	
Improvement	+75.9%	+50.6%	

Table 11: Temporal generalization: train on June 2024, test on September 2024. The model degrades by 30% but retains a 32.8% advantage over the LUT. Ranking quality (ρ) is nearly unchanged. Path coverage drops from 91.6% to 85.1%.

Drift analysis. Of the 3,612 tracker domains in September, 88.2% also appear in the June training set; 11.8% are new. URL path overlap is lower: only 14.9% of unique September paths were seen in June training, though these cover 85.1% of September *rows* (frequent paths recur; rare paths are ephemeral). The transfer size distribution is stable: June and September have identical medians (43 bytes) and similar means (13,621 vs. 14,258).

Results. The model's MAE degrades by 30.3% (3,466 → 4,517) over three months (Table 11). The path LUT degrades by a similar amount (30.8%), while the domain+type LUT barely changes (+1.9%)—its coarser granularity makes it more temporally stable but permanently less accurate.

Critically, the model retains a **32.8% advantage over the LUT** on September data, compared to 47.5% on June. Ranking quality is nearly unchanged (ρ = 0.942 vs. 0.945). The weekly aggregation

estimate degrades from 6.1% to 10.1% median error—still substantially below the LUT's 21.5%. Scripts remain the strongest category, with the model achieving +50.6% improvement in September versus +75.9% in June.

The degradation is driven by URL path churn: only 14.9% of unique September paths were seen in June, compared to the 91.6% coverage within the June random split. This suggests **quarterly retraining** on fresh HTTP Archive data to maintain accuracy, which is feasible given the monthly crawl cadence.

7 Analysis

7.1 Domain-Level Estimation Does Not Need ML

A natural first approach is domain-level scoring: assign each tracker domain a static cost score using aggregated HTTP Archive features. We built this pipeline for 4,592 domains, training XGBoost on 19 features to predict CPU and network cost scores (Spearman ρ = 0.751 and 0.999 respectively).

The problem does not require machine learning. Of the 4,592 domains, 2,185 (48%) have direct Lighthouse CPU measurements and need no prediction. The remaining 2,407 (52%) fall below Lighthouse's ~50 ms reporting threshold and are uniformly inexpensive. KS tests confirm the two populations are disjoint on every feature ($p < 0.001$; median `max_script_bytes`: 20,338 vs. 585). Self-training [23] produced zero improvement because the feature-space disjointness prevented any unlabeled domain from receiving a confident pseudo-label.

This negative result has a constructive implication: it establishes the criteria for a genuine prediction problem in this domain. The domain-level formulation fails the test because labels are available (or trivially assignable) for both populations. The per-request formulation passes: blocked requests have no response, and the URL-level variance (Section 3.3) is too large for lookup tables to resolve.

7.2 Why Tweedie Loss Outperforms Squared Error

The transfer size distribution is zero-inflated: 39.5% of values are exactly zero (beacons), while the remaining 60.5% span five orders of magnitude. Squared error treats a 100-byte error on a beacon identically to a 100-byte error on a 170 KB script. Tweedie loss with $p > 1$ scales the penalty by $\hat{y}^{2-p}$, effectively down-weighting near-zero predictions where absolute error is already small and up-weighting large predictions where accuracy matters for the aggregate.

This explains the 23% MAE gap (3,466 vs. 4,527). It also explains why the gap translates to an even larger aggregation advantage: the beacons that squared error over-optimizes for contribute negligibly to the weekly total, while the scripts that Tweedie prioritizes dominate it. Figure 11 confirms the model tracks the diagonal across five orders of magnitude, with tightest predictions in the 10 KB–100 KB range (JavaScript bundles).

8 Deployment Considerations

Feature availability. When ETP intercepts a request, Firefox's `nsIChannel` and `nsILoadInfo` interfaces expose the request URI (maps to all URL features), content policy type (maps to resource type), loading principal (maps to initiator type), and HTTP method. The TF-IDF vocabulary and SVD projection matrix are static artifacts (~2 MB) shipped alongside the model.

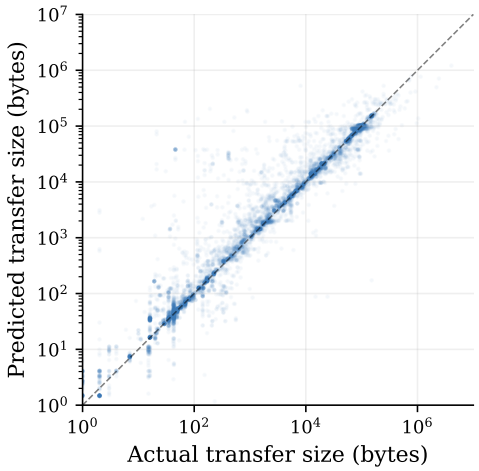

Figure 11: Predicted vs. actual transfer size (log scale, 20K random test samples). The model tracks the diagonal across five orders of magnitude.

Model format. The XGBoost model exports to ONNX format (~500 KB for 500 trees). Inference latency is on the order of microseconds. Since cost estimates are used for background aggregation rather than real-time decisions, latency is not a constraint.

Deployment status. This model is scheduled for release in Firefox in May 2026 as part of a New Tab privacy widget, surfacing per-request cost estimates to approximately 250 million daily active users. Model updates are delivered via Firefox Remote Settings, the same mechanism used for the Disconnect list. Monthly retraining on fresh HTTP Archive crawl data tracks changes in tracker SDK versions and URL structures.

9 Limitations

Temporal stability. The model degrades by 30% over three months (June → September 2024; Section 6.8), driven by URL path churn (only 14.9% of unique September paths were seen in June). The model retains a 32.8% advantage over the LUT on September data, but quarterly retraining on fresh HTTP Archive crawls is recommended for deployment. Longer-term degradation (6–12 months) has not been measured.

Timing metric validity. Transfer size is server-determined and browser-independent. Timing metrics depend on network stack, hardware, and conditions. HTTP Archive uses a fixed test environment (Moto G4, cable connection), so timing predictions are standardized-condition estimates rather than personalized predictions. We report timing results but recommend deploying only the transfer size model.

Training–deployment population shift. The model trains on HTTP Archive's third-party entity list and deploys on Firefox's Disconnect

list. These overlap substantially but not completely. Disconnect-only domains absent from HTTP Archive receive the global median fallback.

Server-side variation. A/B tests, geo-targeting, and SDK versioning cause the same URL to return different payloads. This is irreducible noise: no pre-response feature can predict server-side randomization. Aggregation across many requests mitigates this.

Ecological validity of aggregation simulation. Domain-correlated browsing simulation (Section 6.6) roughly doubles the model's weekly error (6.3% → 12.9%), confirming that error correlation reduces cancellation. Real browsing patterns may differ from our 15-domain simulation in ways we cannot fully characterize without per-user telemetry.

10 Conclusion

We have shown that the performance cost of blocked tracker requests—structurally unobservable because the response is never received—can be predicted from pre-response features with sufficient accuracy for user-facing display. Our analysis of 3,490,824 tracker requests reveals that within-domain transfer size variance spans three orders of magnitude, but this variance is predictable from URL structure: learned models outperform even exact-path memorization by regularizing across similar URL patterns.

Three findings generalize beyond the Firefox deployment context. First, for zero-inflated web response data, loss function selection (Tweedie vs. squared error, 23% gap) matters more than model architecture—a reusable insight for web measurement tasks with similar distributional properties. Second, data-driven TF-IDF URL embeddings subsume hand-crafted regex features, reducing the need for domain-specific feature engineering in URL-based prediction tasks. Third, per-request prediction noise cancels in aggregation even under correlated browsing, enabling accurate cost estimation from inherently noisy individual predictions. These results hold under correlated-browsing simulation, and the framework generalizes to any browser intervention where the cost of the blocked action is unobservable.

References

[1] Andrius Aucinas, Istvan Haller, and Ben Livshits. 2019. Accurately Predicting Ad Blocker Savings. https://brave.com/blog/accurately-predicting-ad-blocker-savings/.
[2] Shai Ben-David, John Blitzer, Koby Crammer, Alex Kulesza, Fernando Pereira, and Jennifer Wortman Vaughan. 2010. A Theory of Learning from Different Domains. *Machine Learning* 79, 1–2 (2010), 151–175.
[3] Léon Bottou, Jonas Peters, Joaquin Quiñonero-Candela, Denis X Charles, D Max Chickering, Elon Portugaly, Dipankar Ray, Patrice Simard, and Ed Snelson. 2013. Counterfactual Reasoning and Learning Systems: The Example of Computational Advertising. *Journal of Machine Learning Research* 14 (2013), 3207–3260.
[4] Ismael Castell-Uroz, Pere Barlet-Ros, and Josep Solé-Pareta. 2020. Demystifying Content-Blockers: A Large Scale Study of Actual Performance Gains. In *Conference on Network and Service Management (CNSM)*.
[5] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 785–794.
[6] Disconnect. 2024. Disconnect Tracking Protection List. https://github.com/disconnectme/disconnect-tracking-protection. Accessed: 2024-06-01.
[7] Ghostery. 2020. Tracker Tax: The Impact of Third-Party Trackers on Website Speed. https://www.ghostery.com/tracker-tax.
[8] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why do tree-based models still outperform deep learning on typical tabular data?. In *Advances in Neural Information Processing Systems*, Vol. 35.
[9] HTTP Archive. 2022. Web Almanac 2022: Third Parties. https://almanac.httparchive.org/en/2022/third-parties.
[10] HTTP Archive. 2024. HTTP Archive: Tracking How the Web is Built. https://httparchive.org.
[11] Fredrik Johansson, Uri Shalit, and David Sontag. 2016. Learning Representations for Counterfactual Inference. In *Proceedings of the 33rd International Conference on Machine Learning*. 3020–3029.
[12] Bent Jørgensen. 1987. Exponential Dispersion Models. *Journal of the Royal Statistical Society: Series B (Methodological)* 49, 2 (1987), 127–145.
[13] Arjaldo Karaj, Sam Macbeth, Rémi Berson, and Josep M Pujol. 2018. WhoTracks.Me: Shedding Light on the Opaque World of Online Tracking. In *arXiv preprint arXiv:1804.08959*.
[14] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, Vol. 30.
[15] Georgios Kontaxis and Monica Chew. 2015. Tracking Protection in Firefox for Privacy and Performance. In *IEEE Workshop on Web 2.0 Security and Privacy (W2SP)*.
[16] Sinno Jialin Pan and Qiang Yang. 2010. A Survey on Transfer Learning. *IEEE Transactions on Knowledge and Data Engineering* 22, 10 (2010), 1345–1359.
[17] Behnam Pourghassemi, Jordan Ayers, Bobby Bhattacharjee, and Michelle L Mazurek. 2021. adPerf: Characterizing the Performance of Third-party Ads. In *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, Vol. 5. 1–26.
[18] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. CatBoost: Unbiased Boosting with Categorical Features. In *Advances in Neural Information Processing Systems*, Vol. 31.
[19] Joaquin Quiñonero-Candela, Masashi Sugiyama, Anton Schwaighofer, and Neil D Lawrence. 2009. *Dataset Shift in Machine Learning*. MIT Press.
[20] Paul R Rosenbaum and Donald B Rubin. 1983. The Central Role of the Propensity Score in Observational Studies for Causal Effects. *Biometrika* 70, 1 (1983), 41–55.
[21] Hidetoshi Shimodaira. 2000. Improving Predictive Inference under Covariate Shift by Weighting the Log-Likelihood Function. *Journal of Statistical Planning and Inference* 90, 2 (2000), 227–244.
[22] Masashi Sugiyama, Matthias Krauledat, and Klaus-Robert Müller. 2007. Covariate Shift Adaptation by Importance Weighted Cross Validation. *Journal of Machine Learning Research* 8 (2007), 985–1005.
[23] David Yarowsky. 1995. Unsupervised Word Sense Disambiguation Rivaling Supervised Methods. In *Proceedings of the 33rd Annual Meeting of the Association for Computational Linguistics*. 189–196.